Programming Project 4

Graded

2 Days, 10 Hours Late

Student

Jinku Jung

Total Points

66 / 70 pts

Autograder Score

0.0 / 30.0

Question 2

Autograder Adjustment

30 / 0 pts

+ 30 pts +20 for compilation, +10 for passing test cases. Make sure to follow the instructions, as it should be able to take in lowercase letters

Question 3

Runtime (Manual)

12 / 12 pts

✔ **+ 2 pts** Asks for bets between 1-5 tokens before hand is shown and does not work if you input more than 5 or less than 1

✔ **+ 2 pts** Shows hand (after betting and after exchanging) and clears hand each round

✔ **+ 2 pts** Exchanges cards correctly

✔ **+ 2 pts** Allows you to play multiple games with updated balance if the user chooses

✔ **+ 2 pts** Implements payouts relative to tokens used

✔ **+ 2 pts** Shuffles deck correctly

Does not shuffle your hand between rounds. Actually he does.

Question 4

Style

10 / 11 pts

Header

✔ **+ 2 pts** Complete header with name, uni and description

Comments

✔ **+ 2 pts** Descriptive and concise comments throughout code

Variable Names

✔ **+ 1 pt** Descriptive variable names

Line Length

✔ **+ 2 pts** All lines 80 characters or less.

Whitespace

✔ **+ 1 pt** Some whitespace between methods or logical blocks of code. Somewhat difficult to read, but clear break between different methods

Proper Indentation

✔ **+ 2 pts** All code is properly indented, allowing for readability.

A lot of comments that should probably just be in the readMe instead, but not deducting points. Some of your method names are a bit unusual and use snake_case rather than camelCase, but no points deducted. For your switch statements, each statement should be on it's own line, so case 1: output += "C"; break; should be three lines.

Question 5

Design

8 / 10 pts

Instance variables

- ✔ **+ 1 pt** instance variables all private

compareTo()

- ✔ **+ 2 pts** Compares the suit and value of the card

toString()

- ✔ **+ 2 pts** Displays card in a human readable format (e.g., s4, 4s, Four of Spades, 4 of spades but does not translate 12 to Queen etc...)

Command Line Args work

- ✔ **+ 2 pts** Starts a game using specified hand or even just prints the hand with the given cards in the player's hand -

Helper Methods

- ✔ **+ 0 pts** Some helper methods are public (okay for checkHand to be public)
- ✔ **+ 1 pt** checkhand uses helper methods for each type of hand

- the individual check methods are helpers and should be private!

Question 6

Je ne sais quoi

6 / 7 pts

Encapsulation

- ✔ **+ 1 pt** Player has no references to Deck, Card, or Game.
- ✔ **+ 1 pt** Deck should only have knowledge of the Card class. It should not have a reference to Player or Game.
- ✔ **+ 1 pt** Game is the only class with knowledge of all the other classes (i.e. it knows about Player and Deck and uses Cards)

- ✔ **+ 2 pts** Code is broken up into logical blocks, eg. methods for checking each type of hand or dealing to the player
- ✔ **+ 1 pt** compareTo returns 0 when both suit and rank are the same.
- Print statement in Card (all print statements should be in Game)

Question 7

Readme and Format

0 / 0 pts

✔ **+ 0 pts** Contains readme

Autograder Results

The autograder failed to execute correctly. Please ensure that your submission is valid. Contact your course staff for help in debugging this issue. Make sure to include a link to this page so that they can help you most effectively.

Submitted Files

Card.java Download

```
/* Name: Jinku Jung
 * UNI: dj2598
 * Assignment: Programming Project #4
 * Date: 3/29/2021
 */


/* File: Card.java
 *
 *  Purpose: This is the Card class file for the Video Poker game.
 *  Card.java can create custom Card objects, encoding a card's:
 *
 *      1.) Suit as an integer value from 1 through 4.
 *
 *      2.) Rank as an integer value from 1 through 13.
 *
 *  Instance Variables:
 *
 *      1.) int suit = the numeric equivalent of a Card object's
 *          suit.
 *
 *      2.) int rank = the numeric equivalent of a Card object's
 *          ordinal rank as printed on the card AND 11 through
 *          13 for the Jack, Queen, and King, respectively.
 *
 *  Constructor: Instantiates exactly one Card object when
 *              invoked.
 *
 *  Methods: The Card class has six methods defined:
 *
 *      1.) public int compareRank(Card c)
 *
 *      2.) public int compareSuit(Card c)
 *
 *      3.) public int compareTo(Card c)
 *
 *      4.) public int getRank()
 *
 *      5.) public int getSuit()
 *
 *      6.) public String toString()
 */


//Imported from VS Code on 3/29/2021 @ 11:42 AM EST

public class Card implements Comparable<Card> {

    // Instance Variables:
```

```
50
51      private int suit; /* Use integers 1-4 to encode the suit
52                        * (1 = Clubs, 2 = Diamonds, 3 = Hearts;
53                        *  4 = Spades)
54                        */
55      private int rank; /* Use integers 1-13 to encode the rank
56                        * (1 through 13 for: Ace through 10 AND
57                        *  Jack, Queen, & King)
58                        */
59
60      // Constructor:
61
62      public Card(int s, int r)
63      {
64         this.rank = r;
65         this.suit = s;
66      }
67
68      //Methods (in alphabetical order):
69
70      /* Method: CompareRank(Card c) is a decomposed method from the
71       *       "compareTo(Card c)" calling method.  It simply
72       *       compares your implicit Card object parameter to the
73       *       explicit "Card c" input parameter via their integer
74       *       rank, then by their integer suit ONLY AS A
75       *       TIE-BREAKER for comparison.  It returns an integer
76       *       value.
77       */
78
79      public int compareRank(Card c)
80      {
81         if(getRank() < c.getRank())
82         {
83            return -1;
84         }
85         if(getRank() > c.getRank())
86         {
87            return 1;
88         }
89
90         if(getRank() == c.getRank())
91         {
92            return compareSuit(c);
93         }
94
95         return 0;
96      }
97
98      /* Method: CompareSuit(Card c) is the TIE-BREAKER decomposed method
99       *       called exclusively within the compareRank(Card c) outer
100      *       method.  It returns an integer value.
101      */
```

```
102
103     public int compareSuit(Card c)
104     {
105        if(getSuit() < c.getSuit())
106        {
107           return -1;
108        }
109
110        if(getSuit() > c.getSuit())
111        {
112           return 1;
113        }
114
115        /*
116         * You must allot for the case that two exactly
117         * identical cards are found in the deck:
118         */
119
120        return 0;
121     }
122
123     /* Method: CompareTo(Card c) establishes card order first by
124      *        rank and then by suit--only as needed.  It returns an
125      *        integer value.
126      *
127      *        NOTE: This method is used explicitly inside the
128      *        Collections.sort(hand) java library command in our sort()
129      *        method from the Game class (Cannon).
130      */
131
132     public int compareTo(Card c)
133     {
134        return compareRank(c);
135     }
136
137     /* Method: GetRank() is a simple accessor method that returns the "rank"
138      *        instance variable of the implicit Card-object parameter.
139      *        It returns an integer value.
140      */
141
142     public int getRank()
143     {
144        return this.rank;
145     }
146
147     /* Method: GetSuit() is a simple accessor method that returns the "suit"
148      *        instance variable of the implicit Card-object parameter.
149      *        It returns an integer value.
150      */
151
152     public int getSuit()
153     {
```

```
154         return this.suit;
155     }
156
157     /* Method: SetRank(int j) is a simple mutator method that sets the
158      *       "rank" instance variable to the input value of "j."
159      *       It returns nothing.
160      */
161
162     public void setRank(int j)
163     {
164         this.rank = j;
165     }
166
167     /* Method: SetSuit(int i) is a simple mutator method that sets the
168      *       "rank" instance variable to the input value of "i." It
169      *       returns nothing.
170      */
171
172     public void setSuit(int i)
173     {
174         this.suit = i;
175     }
176
177     /* Method: ToString() is a simple method that converts the integer
178      *       values for suit and rank inside a given Card object into
179      *       a two/three (ex: S10) character string for its return type.
180      *
181      *       NOTE: This sub-method is used to display the card in
182      *             "output form" for the showHand(hand)
183      *             method (from the Game class) to display to the user.
184      */
185
186     public String toString()
187     {
188         //Instantiate "output" as a String object.
189
190         String output = "";
191
192         // Append "suit" as a one-character string to "output."
193
194         switch(this.getSuit())
195         {
196             case 1: output += "C"; break;
197
198             case 2: output += "D"; break;
199
200             case 3: output += "H"; break;
201
202             case 4: output += "S"; break;
203
204             default: System.out.println("Invalid suit assigned "
205                                 + "to this Card object!");
```

```
206                 break;
207         }
208
209         // Append "rank" to the "output" string.
210
211         switch(this.getRank())
212         {
213             case 1: output += "A "; break;
214
215             case 11: output += "J "; break;
216
217             case 12: output += "Q "; break;
218
219             case 13: output += "K "; break;
220
221             default: output += this.rank + " "; break;
222         }
223
224         // Return the final card "output" to display.
225
226         return output;
227     }
228 }
229
```

Deck.java Download

```
/* Name: Jinku Jung
 * UNI: dj2598
 * Assignment: Programming Project #4
 * Date: 3/29/2021
 */


/* File: Deck.java
 *
 *  Purpose: This is the Deck class file for the Video Poker game.
 *  Deck.java is responsible for generating and populating our
 *  standard 52-card Poker card deck--in a fixed-sized array of
 *  Card objects.
 *
 *  Instance Variables:
 *
 *      1.) Card[] cards: This object reference holds all
 *          the cards in the poker game's deck.
 *
 *      2.) int top: this primitive data type just keeps
 *          track of where the next card to be dealt from the
 *          deck actually is and can only be incremented AND
 *          NEVER decremented.  In one-player Video Poker, this
 *          value will never reach the end of the deck
 *          (the 52nd card) since a player can only be dealt 5
 *          initial cards and can replace (and thus be dealt an
 *          additional # of cards) up to 5 more cards, from
 *          the top of the deck.
 *
 *          NOTE: This happens whenever the "discard(Scanner
 *          input)" method from the Game class is invoked.
 *
 *  Constructor: Deck(), when invoked, will contain two items:
 *
 *      1.) Card[] cards: an array exclusively of Card-type
 *          objects.
 *
 *      2.) A integer variable called "top," to denote the
 *          current position of the next card that can be
 *          drawn from the deck.  Once shuffled, the order
 *          of the Card objects in cards is immutable and
 *          serves as a reference for cards already drawn
 *          and cards yet to be drawn in a single Video
 *          Poker game.
 *
 *  Methods:
 *
 *      1.) public Card deal(): just deals one card off from the "top" of
 *          the deck and returns a Card object back to its
```

```
50  *         calling implicit parameter.
51  *
52  *      2.) public void incrementTop(): to keep track of the next card to
53  *         be drawn from the deck.  Also, it denotes which
54  *         cards were already used/discarded in a single
55  *         Video Poker game.
56  *
57  *      3.) public void shuffle(): pseudo-randomly shuffles the deck when
58  *         invoked.
59  */
60
61
62  //Imported from VS Code on 3/29/2021 @ 11:42 AM EST
63
64  import java.util.Random;
65
66  public class Deck {
67
68     //Instance Variables:
69
70     private Card[] cards;
71     private int top; //the index of the top of the deck
72     private final int deckSize = 52;
73
74     /* Constructor: Deck() sets up and populates a complete 52-card
75      *          Poker deck with four suits (1 = Clubs, 2 = Diamonds,
76      *          3 = Hearts, and 4= Spades) and thirteen ranks
77      *          (Ace = 1, 2-10, Jack = 11, Queen = 12, & King = 13)
78      *          in a Card-object array called "cards".
79      *
80      *          Also, the integer variable "top" is created as a
81      *          placeholder for the next card eligible to be dealt
82      *          from the deck.
83      *
84      *          Lastly, deckSize was created as a constant "final"
85      *          integer value, denoting the size of our Video Poker
86      *          game--per deck, per game.
87      */
88
89     public Deck() {
90
91        int suit = 0;
92        int rank = 0;
93
94        cards = new Card[deckSize];
95
96        int i = 0;
97
98           for (suit = 1; suit <= 4; suit++)
99           {
100             for (rank = 1; rank <= 13; rank++)
101             {
```

```
102                 cards[i++] = new Card(suit, rank);
103             }
104         }
105       top = 0;
106   }
107
108       //Methods (in alphabetical order):
109
110   /* Method: Deal() is a no-argument accessor method which
111    *       returns the "top" card from the deck.  IncrementTop is
112    *       called upon to then advance the index of the next
113    *       eligible deck card to be drawn.  It returns a Card object
114    *       value.
115    */
116
117   public Card deal()
118   {
119       Card nextCard = cards[top];
120       incrementTop();
121       return nextCard;
122   }
123
124   /* Method: IncrementTop() is a no-argument mutator method which
125    *       increments the "top" counter variable for the deck's next
126    *       eligible card to be drawn.  It returns nothing.
127    */
128
129   public void incrementTop()
130   {
131       top++;
132   }
133
134   /* Method: Shuffle() is a no-argument mutator method which
135    *       randomly re-orders the card deck, using a pseudorandom seed
136    *       value generated from the Math.random Java library method.
137    *       It returns nothing.
138    */
139
140   public void shuffle()
141   {
142     /* From <http://www.mathcs.emory.edu/~cheung/Courses/170/Syllabus/10/
143      * deck-of-cards.html> accessed on 3/29/2021 @ 12:56 PM EST.
144      *
145      * See my readMe.txt for the full "shuffle()" algorithm source credit.
146      */
147
148       int i, j, k;
149
150           for ( k = 0; k < cards.length; k++ )
151           {
152           //Select two randomly-positioned cards from the deck.
153           i = (int) ( deckSize * Math.random() );
```

```
154             j = (int) ( deckSize * Math.random() );
155 
156                 // Use a dummy variable "tempCard" to help swap these cards.
157 
158             Card tempCard = cards[i];
159           cards[i] = cards[j];
160             cards[j] = tempCard;
161           }
162       top *= 0;
163     }
164 }
```

Game.java Download

```
/* Name: Jinku Jung
 * UNI: dj2598
 * Assignment: Programming Project #4
 * Date: 3/29/2021
 */

/* File: Game.java
 *
 *  Purpose: This is the Game class file for the Video Poker game.
 *  Game.java is responsible for declaring, instantiating, and
 *  executing all top-level Video Poker methods required to run
 *  the game.  Game has five instance variables:
 *
 *
 *      1.) Player p: This object reference allows for the
 *          Player() constructor to be called to set up the
 *          user's money methods germane to Video Poker.
 *
 *      2.) Deck cards: this object reference allows Deck
 *          methods to be called inside the Game class plus
 *          indirect access to Deck class instance variables.
 *
 *      3.) ArrayList<Card> hand: this ArrayList of Card
 *          objects is responsible for setting, holding, and
 *          retrieving the individual five cards of a player's
 *          Video Poker hand.
 *
 *      4.) Scanner input: this object reference holds whatever
 *          user keyboard inputs are requested and assigned to
 *          this variable via the Scanner interface.
 *
 *      5.) int Choice: this prmitive data type's sole purpose
 *          is to act as a "yes or no" placeholder that determines
 *          whether or not to run one Video Poker game at a
 *          time--governed by the play() method's "do ... while"
 *          loop.
 *
 *  Constructors: Game class has two constructors, per the skeleton
 *  provided by Professor Cannon:
 *
 *      1.) The normal Game() constructor: used for real Video Poker
 *          games with a human player.
 *
 *      2.) The overloaded Game(foo) constructor: used for simulated
 *          Video Poker games, fed in by a statically-declared
 *          String[] testHand array as an input parameter.
 *          Used primarily for debugging the checkHand(hand)
 *          method from this Game class.
 *
```

```
50  * Methods: Game class contains 32 total methods:
51  *
52  *     1.) public Boolean checkFlush(ArrayList<Card> hand)
53  *
54  *     2.) public Boolean checkFourOfAKind(ArrayList<Card> hand)
55  *
56  *     3.) public Boolean checkFullHouse(ArrayList<Card> hand)
57  *
58  *     4.) public int checkHand(ArrayList<Card> hand)
59  *
60  *     5.) public Boolean checkOnePair(ArrayList<Card> hand)
61  *
62  *     6.) public Boolean checkRoyalFlush(ArrayList<Card> hand)
63  *
64  *     7.) public Boolean checkStraight(ArrayList<Card> hand)
65  *
66  *     8.) public Boolean checkStraightFlush(ArrayList<Card> hand)
67  *
68  *     9.) public Boolean checkThreeOfAKind(ArrayList<Card> hand)
69  *
70  *     10.) public Boolean checkTwoPair(ArrayList<Card> hand)
71  *
72  *     11.) public void dealHand()
73  *
74  *     12.) public void discard(Scanner input)
75  *
76  *     13.) public void evaluateHand(ArrayList<Card> hand)
77  *
78  *     14.) public void exitGame()
79  *
80  *     15.) public int is_A_Bust()
81  *
82  *     16.) public Boolean isBankrupt()
83  *
84  *     17.) public void loadHand_Via_TestHand(int i, int suit,
85  *                                            int rank,
86  *                                            String[] testHand,
87  *                                            ArrayList<Card> hand)
88  *
89  *     18.) public void manageBankroll (Scanner input)
90  *
91  *     19.) public int multiplicity(int index)
92  *
93  *     20.) public void placeGameBet (Scanner input)
94  *
95  *     21.) public void play()
96  *
97  *     22.) public void showBankroll()
98  *
99  *     23.) public int showGameMenu(Scanner input)
100 *
101 *     24.) public void showHand(ArrayList<Card> hand)
```

```
102  *
103  *      25.) public int show_Keep_Or_Discard(int i, Scanner input)
104  *
105  *      26.) public void shuffleDeck(Deck cards)
106  *
107  *      27.) public void sort(ArrayList<Card> hand)
108  *
109  *      28.) public int suitType(String letterSuit)
110  *
111  *      29.) public void testHand_Error_Message(int i)
112  *
113  *      30.) public void testPlay()
114  *
115  *      31.) public void testResult(int odds, String typeOfHand,
116  *                                  ArrayList<Card> hand)
117  *
118  *      32.) public String which_Type_Of_Hand (int odds, String typeOfHand)
119  *
120  */
121
122
123 //Imported from VS Code on 3/29/2021 @ 11:42 AM EST
124
125 import java.util.ArrayList;
126 import java.util.Collections;
127 import java.util.Scanner;
128
129
130 public class Game {
131
132    // Instance Variables:
133
134    private Player p;
135    private Deck cards;
136    private ArrayList<Card> hand;
137    private Scanner input;
138    private int choice;
139
140
141       // Constructors:
142
143    // This no-argument constructor is to actually play a normal game.
144
145    public Game()
146    {
147       // Setup game cards.
148       cards = new Deck();
149
150       // Setup player data.
151       p = new Player();
152
153       // Set the top-level game option variable to "play."
```

```
154         choice = 1;
155
156         // Initialize the ArrayList<Card> hand object.
157         hand = new ArrayList<Card>();
158
159         // Initialize the Scanner input object.
160         input = new Scanner(System.in);
161         }
162
163     /* This overloaded version of the Game(foo) constructor is chiefly used
164      * for debugging purposes, especially pertaining the validity of
165      * your "public int checkHand(ArrayList<Card> hand)" method of
166      * this Game class.
167      */
168
169     public Game(String[] testHand)
170     {
171         int rank = 0;
172         int suit = 0;
173
174         hand = new ArrayList<Card>();
175
176
177         for (int i = 0; i < 5; i++)
178         {
179             String letterSuit = testHand[i].substring(0, 1);
180
181             suit = suitType(letterSuit);
182
183             if(suit > 0)
184             {
185                 loadHand_Via_TestHand(i, suit, rank, testHand, hand);
186
187             }
188             else
189             {
190                 testHand_Error_Message(i);
191             }
192         }
193     }
194
195     // Methods (in alphabetical order):
196
197     /* Method: CheckFlush(ArrayList<Card> hand) takes in the player's
198      *       five card Poker hand to evaluate it as a possible Flush:
199      *       a hand where all five cards are of the exact same suit,
200      *       yet not in consecutive order.  It returns a Boolean value.
201      */
202     public Boolean checkFlush(ArrayList<Card> hand)
203     {
204
205         Card myCard = hand.get(0);
```

```
206
207         for (int i = 1; i < 5; i++)
208         {
209             Card nextCard = hand.get(i);
210 
211             if( myCard.getSuit() != nextCard.getSuit() )
212             {
213                 return false;
214             }
215         }
216         return true;
217     }
218 
219     /* Method: CheckFourOfAKind(ArrayList<Card> hand) uses the
220     *         "multiplicity(int i)" method to count repeated card
221     *         ranks in a given hand. This method verifies a winning
222     *         Video Poker game condition.  It returns a Boolean value.
223     */
224     public Boolean checkFourOfAKind(ArrayList<Card> hand)
225     {
226 
227         for(int i = 0; i < 5; i++)
228         {
229 
230             if(multiplicity(i) == 4)
231             {
232                 System.out.println("\nPlayer draws Four Of A Kind!\n");
233                 return true;
234             }
235         }
236         return false;
237     }
238 
239     /* Method: CheckFullHouse(ArrayList<Card> hand) uses the
240      *         "checkThreeOfAKind(hand)"-->"checkOnePair(hand)" sequence
241      *         of method calls to verify this winning Video Poker game
242      *         condition.  It returns a Boolean value.
243      */
244 
245     public Boolean checkFullHouse(ArrayList<Card> hand)
246     {
247         if(checkThreeOfAKind(hand))
248         {
249             if(checkOnePair(hand))
250             {
251                 System.out.println("\nPlayer draws a Full House!\n");
252                 return true;
253             }
254             return false;
255         }
256         return false;
257     }
```

```
258
259     /* Method: CheckHand(ArrayList<Card> hand) takes in an ArrayList
260      *        of five Card objects and evaluates the hand.  Win or
261      *        lose, the player's results are displayed onto the
262      *        Terminal screen as user output.  It returns an integer
263      *        value, to be assigned to the "odds" integer variable
264      *        found in my "evaluateHand(ArrayList<Card> hand)" Game
265      *        class method.
266      *
267      *        NOTE: This method is also heavily decomposed
268      *              for code readability purposes.
269         */
270
271     public int checkHand(ArrayList<Card> hand)
272     {
273        sort(hand);
274
275        if(checkRoyalFlush(hand)) {return 250;}
276
277        if(checkStraightFlush(hand)) {return 50;}
278
279        if(checkFourOfAKind(hand)) {return 25;}
280
281        if(checkFullHouse(hand)) {return 6;}
282
283        if(checkFlush(hand))
284        {
285           System.out.println("\nPlayer draws a Flush!\n");
286           return 5;
287        }
288
289        if(checkStraight(hand))
290        {
291           System.out.println("\nPlayer draws a Straight!\n");
292           return 4;
293        }
294
295        if(checkThreeOfAKind(hand))
296        {
297           System.out.println("\nPlayer draws Three Of A Kind!\n");
298           return 3;
299        }
300
301        if(checkTwoPair(hand))
302        {
303           System.out.println("\nPlayer draws Two Pair!\n");
304           return 2;
305        }
306
307        if(checkOnePair(hand))
308        {
309           System.out.println("\nPlayer draws One Pair!\n");
```

```
310         return 1;
311       }
312
313       return is_A_Bust();
314     }
315
316     /* Method: CheckOnePair(ArrayList<Card> hand) is a method which
317      *       simply checks for the existence of one pair of
318      *       equal-ranked cards in the player's hand.  It returns
319      *       a Boolean value.
320      */
321
322     public Boolean checkOnePair(ArrayList<Card> hand)
323     {
324       for (int i = 0; i < 5; i++)
325       {
326         if(multiplicity(i) == 2)
327         {
328           return true;
329         }
330       }
331       return false;
332     }
333
334     /* Method: CheckRoyalFlush(ArrayList<Card> hand) verifies if the
335      *       player holds the "10, Jack, Queen, King, Ace" magical
336      *       winning hand in Video Poker.  It returns a Boolean value.
337      */
338
339     public Boolean checkRoyalFlush(ArrayList<Card> hand)
340     {
341       // Checks to see if all cards in the hand are of the same suit.
342       if( checkFlush(hand) )
343       {
344         // Verifies the 1st card of the sorted hand is actually
345         // the Ace.
346         if(hand.get(0).getRank() == 1)
347         {
348
349           // Ensures the 2nd through 5th card are in ascending &
350           // consecutive order AND each of those cards are either
351           // the "10" or "Jack" or "Queen" or "King."
352
353           for(int i = 1; i < 4; i++)
354           {
355
356             if( ( (hand.get(i).getRank() + 1)
357                   != hand.get(i+1).getRank() )
358                     || ( hand.get(i).getRank() < 10) )
359             {
360               // Somehow, the player is missing the required
361               // "10, J, Q, K" sequence for cards 2 through
```

```
                // 5 --> his/her hand is NOT a Royal Flush.
                return false;
            }
          }
          // Thus, the player has a hand of "A, 10, J, Q, K" that's
          // all in the same suit --> a Royal Flush is detected!
          System.out.println("\nPlayer draws a Royal Flush!\n");
          return true;
        }
        // Player doesn't even have the Ace in his hand --> his/her
        // hand is NOT a Royal Flush.
        return false;
    }
    // Player's hand is not even a Flush --> his/her hand is NOT a
    // Royal Flush.
    return false;
}

/* Method: CheckStraight(ArrayList<Card> hand) verifies if the player
 *       did indeed draw five consecutive cards, not of the same
 *       suit.  It returns a Boolean value.
 */

public Boolean checkStraight(ArrayList<Card> hand)
{
    // Checks if the 1st card of the sorted hand is actually
    // an Ace.
    if(hand.get(0).getRank() == 1)
    {

        // Ensures the 2nd through 5th card are in ascending &
        // consecutive order AND each of those cards are either
        // the "10" or "Jack" or "Queen" or "King."

        for(int i = 1; i < 4; i++) {

            // If cards 2-5 aren't "10, J, Q, K" then check to
            // see if the Ace is in a "non-face card" straight:

            if( ( (hand.get(i).getRank() + 1)
                  != hand.get(i+1).getRank() )
                      || ( hand.get(i).getRank() < 10) )
            {
               for(int j = 0; j < 4; j++)
               {
                  if( ( (hand.get(j).getRank() + 1)
                       != hand.get(j + 1).getRank() ) )
                  {
                     // If a non-consecutive non-face card is
                     // present in the entire hand, then the
                     // player does NOT have a straight.
                     return false;
```

```
414                 }
415               }
416               //Thus, the player has a "A, 2, 3, 4, 5" straight:
417               return true;
418            }
419         }
420         // If the player DOES have "10, J, Q, K" for cards 2-5,
421         // then he/she MUST have a "face card" straight of
422         // "A, 10, J, Q, K" based upon each card's rank and order.
423         return true;
424      }
425
426      else
427
428      //Now just check for a straight, sans the Ace:
429      {
430
431         for(int i = 0; i < 4; i++)
432         {
433            if( ( (hand.get(i).getRank() + 1)
434                != hand.get(i + 1).getRank() ) )
435            {
436               // If a non-consecutive card is present,
437               // then the player does NOT have a straight.
438               return false;
439            }
440         }
441         // If all 5 cards in the hand are consecutive in when
442         // sorted, then the player has a straight.
443         return true;
444      }
445   }
446
447   /* Method: CheckStraightFlush(ArrayList<Card> hand) checks if the
448    *       player did indeed draw a flush AND straight, in that
449    *       exact order for this winning hand.  Else, the method is
450    *       false.  It returns a Boolean value.
451    */
452
453   public Boolean checkStraightFlush(ArrayList<Card> hand)
454   {
455
456      if(checkFlush(hand))
457      {
458         if(checkStraight(hand))
459         {
460            System.out.println("\nPlayer draws a Straight Flush!\n");
461            return true;
462         }
463         return false;
464      }
465      return false;
```

```
466     }
467
468     /* Method: CheckThreeOfAKind(ArrayList<Card> hand) uses the
469      *       "multiplicity(int i)" method to detect the number
470      *       of equal-ranked cards with repeated occurences
471      *       through out the 5 card Video Poker hand.  It returns
472      *       a Boolean value.
473      */
474
475     public Boolean checkThreeOfAKind(ArrayList<Card> hand)
476     {
477
478        for(int i = 0; i < 5; i++) {
479           if(multiplicity(i) == 3) {
480              return true;
481           }
482        }
483        return false;
484     }
485
486     /* Method: CheckTwoPair(ArrayList<Card> hand) again uses the
487      *       "multiplicity(int i)" method to detect two
488      *       even-ranked pairs whereby the first pair's rank
489      *       does NOT equal the second pair's rank.  Else, in this
490      *       special case, it should have been scored as a "Four
491      *       of a Kind" winning hand in a previous line-call inside
492      *       the parent checkHand(hand) calling method.
493      */
494
495     public Boolean checkTwoPair(ArrayList<Card> hand) {
496
497        for(int i = 0; i < 5; i++) {
498
499           for (int j = i + 1; j < 5; j++)
500           {
501              if ( (multiplicity(i) == 2) && (multiplicity(j) == 2) &&
502                 ( hand.get(i).getRank() != hand.get(j).getRank() ) )
503              {
504                 return true;
505              }
506           }
507        }
508        return false;
509     }
510
511     /* Method: DealHand() deals the player his initial 5 cards for
512      *       Video Poker for the showHand(hand) method to display
513      *       onto the user's terminal screen.  It returns nothing.
514      */
515
516     public void dealHand()
517     {
```

```
518         System.out.println("\n" +
519            "=======================================================");
520         System.out.println("The Deal");
521         System.out.println("" +
522            "=======================================================");
523
524         System.out.println("\nDealing your hand now . . .");
525
526         for (int i = 0; i < 5; i++)
527         {
528            hand.add( cards.deal() );
529            cards.incrementTop();
530         }
531
532         sort(hand);
533      }
534
535      /* Method: Discard(Scanner input) is the parent method which governs
536       *       the "keep or discard" your card process in Video Poker.
537       *       Depending on the user's input, he/she can choose to replace
538       *       up to 5 cards from their originally dealt hand.  They MUST
539       *       make ALL their keep/discard selections first before the
540       *       hand is actually scored by Video Poker in the
541       *       evaluateHand(hand) method.  It returns nothing.
542       */
543
544       public void discard(Scanner input)
545      {
546
547         for (int i = 0; i < 5; i++)
548         {
549            if (hand.get(i) != null)
550            {
551               int discard_Choice = show_Keep_Or_Discard(i, input);
552
553               if (discard_Choice == 0)
554               {
555                  Card newCard = cards.deal();
556                  hand.set(i, newCard);
557               }
558            }
559         }
560         sort(hand);
561      }
562
563      /* Method: EvaluateHand(ArrayList<Card> hand) is the parent method that
564       *       actually scores the user's Video Poker hand for a possible
565       *       payout, based upon their bet amount and winning hand odds.
566       *       EvaluateHand is invoked by the more abstract play() method.
567       *       It returns nothing.
568       */
569      public void evaluateHand(ArrayList<Card> hand)
```

```
570     {
571        int odds = checkHand(hand);
572
573        if(odds > 0)
574        {
575           System.out.println("\n" +
576           "=========================================================");
577           System.out.println("Winner, winner, chicken dinner!");
578           System.out.println("" +
579           "=========================================================");
580
581           p.winnings(odds);
582           System.out.println("You won " + (p.getBet() * odds) +
583                       " tokens!");
584           System.out.println("" +
585           "=========================================================");
586        }
587
588        if(odds == 0 )
589        {
590           System.out.println("\n" +
591           "=========================================================");
592           System.out.println("You lose.");
593           System.out.println("You are now " + (p.getBet()) +
594                       " tokens all the poorer ...");
595           System.out.println("" +
596           "=========================================================");
597        }
598     }
599
600     /* Method: ExitGame() is a screen prompting method that just outputs
601      *       a "good-bye" message to the Video Poker user when he
602      *       chooses to or is forced to quit the game, depending on the
603      *       context of where & where it is invoked.  It returns nothing.
604      */
605
606     public void exitGame()
607     {
608        System.out.println("\n" +
609           "=========================================================");
610        System.out.println("Happy trails, stranger!");
611        System.out.println("" +
612           "=========================================================");
613
614        System.out.println("\nHey, thanks for playing Video Poker!");
615        if(p.getBankroll() > 0)
616        {
617           System.out.println("Your final payout is "
618                       + p.getBankroll() + " tokens.");
619        }
620        System.out.println("                    --Jinku Jung\n");
621     }
```

```
622
623     /* Method: Is_A_Bust(), part of the parent checkHand(hand) method,
624      *       simply tells the user he did NOT have a winning hand
625      *       in his Video Poker game and generates null odds for its
626      *       overall calling method, evaluateHand(hand) where
627      *       checkHand(hand) is its "sub-method."  It returns an
628      *       integer value.
629      */
630
631     public int is_A_Bust()
632     {
633        System.out.println("\nPlayer draws a BUST! "
634                        + " Better luck next time, eh?");
635        return 0;
636     }
637
638     /* Method: IsBankrupt() is a method, when invoked, that simply
639      *       checks to see if the player's bankroll balance is not
640      *       zero.  If zero, the user is told he/she is bankrupt
641      *       AND the exitGame() method is thrown.  Else, the
642      *       user may play a game of Video Poker again.  It
643      *       returns a Boolean value.
644      */
645
646     public Boolean isBankrupt()
647     {
648        if (p.getBankroll() < 1)
649        {
650           System.out.println("\n" +
651           "=========================================================");
652           System.out.println("BANKRUPT!");
653           System.out.println("" +
654           "=========================================================");
655           System.out.println("Sorry bud, you gotta go now.");
656           exitGame();
657           return true;
658        }
659        return false;
660     }
661
662     /* Method: LoadHand_Via_TestHand(int i, int suit, int rank,
663      *       String[] testHand, ArrayList<Card> hand) is a very
664      *       short method that just loads a card values,
665      *       originally expressed as a two/three character string,
666      *       into a new Card object to populate the ArrayList called
667      *       "hand" within the overloaded Game(foo) constructor.  "Hand"
668      *       is then inputted into the checkHand(hand) method, once
669      *       it is invoked by the testPlay() method for debugging
670      *       purposes.  It returns nothing
671      */
672
673     public void loadHand_Via_TestHand(int i, int suit, int rank,
```

```
                        String[] testHand,
                        ArrayList<Card> hand)
{
   String stringRank = testHand[i].substring(1);
   rank = Integer.parseInt(stringRank);

   hand.add( ( new Card(suit, rank) ) );
}

/* Method: ManageBankroll(Scanner input) is a method that prompts
 *       the player to add tokens before they first play a
 *       single Video Poker game or enter in a bet for that game.
 *       This is done via a method call to setBankroll(addedTokens)
 *       from the "Player p" object defined in the Game()
 *       constructor.  It returns nothing.
 */

public void manageBankroll (Scanner input)
{
   System.out.println("\n" +
      "=======================================================");
   System.out.println("Manage Your Bankroll");
   System.out.println("" +
      "=======================================================");

   System.out.println("\nEnter the number of tokens you wish \n" +
                      "to add to your Video Poker bankroll.");


   int addedTokens = input.nextInt();

   if(addedTokens > 0)
   {
      p.setBankroll(addedTokens);
   }

   showBankroll();
}

/* Method: Multiplicity(int index) examines the card at the given
 *       index, then returns the number of times that card's rank is
 *       by any and all Card objects active in the player's 5-card
 *       ArrayList named "hand."   This sub-method is recycled by
 *       various winning hand validation methods found within the
 *       checkHand(hand) method.  It returns an integer value.
 */

public int multiplicity(int index)
{

   int repeats = 0;

```

```
726         Card myCard = hand.get(index);
727 
728         for (int j = 0; j < 5; j++) {
729 
730             Card nextCard = hand.get(j);
731 
732             if( myCard.getRank() == nextCard.getRank() ) {
733                 repeats++;
734             }
735         }
736         return repeats;
737     }
738 
739     /* Method: PlaceGameBet(Scanner input) prompts the user each game to
740      *        place their bet of 1-5 tokens per single Video Poker game.
741      *        If the bet exceeds the player's current token balance,
742      *        they are kindly prompted to only bet what they can afford.
743      *        It returns nothing.
744      */
745 
746     public void placeGameBet (Scanner input)
747     {
748 
749         Boolean is_A_Bad_Bet = true;
750         Boolean is_A_Good_Bet = false;
751 
752 
753         int badBet = 0;
754         int goodBet = 0;
755 
756         while( badBet == 0 && goodBet == 0 )
757         {
758             System.out.println("\n" +
759             "=======================================================");
760             System.out.println("Placing Your Bet");
761             System.out.println("" +
762             "=======================================================");
763 
764             System.out.println("\nPlease enter your bet of 1 to 5 " +
765                     "tokens to play a game of Video Poker!");
766 
767             int amt = input.nextInt();
768 
769             if((amt >= 1 && amt <=5) && amt <= p.getBankroll())
770             {
771                 p.bets(amt);
772                 p.setBankroll(-amt);
773 
774                 showBankroll();
775 
776                 goodBet++;
777             }
```

```
778         else
779         {
780            System.out.println("Sorry bud, you can't bid what "
781                  + "you don't have!");
782
783            showBankroll();
784         }
785      }
786   }
787
788   /* Method: Play() is the central governing method in charge of
789    *       running each game of Video Poker.  It returns nothing.
790    *
791    *       Sub-methods were initally created then actually made
792    *       into working code later in this project.
793    *
794    *       In the end, I managed to pare the Play() method down
795    *       to a very reasonable length of 15 actual command lines.
796    *
797    *       NB: A big thanks to my old CS106A Professor, Eric Roberts,
798    *       and my old Section TA, Keith Richards, for riding me
799    *       very hard over coding style issues each and every
800    *       Interactive Grading session I ever had.  I think I
801    *       eventually got the point!
802    */
803
804   public void play()
805   {
806      do
807      {
808         // Ask the player if he/she wants to play or quit.
809         choice = showGameMenu(input);
810
811         // If the player wishes to quit, payout and say goodbye.
812         if(choice == 0)
813         {
814            exitGame();
815         }
816
817         if (choice == 1)
818         {
819            // First, let the player add to his/her token balance.
820            manageBankroll(input);
821
822            // Now, let the player place his bet before a game begins.
823            placeGameBet(input);
824
825            // Shuffle the 52-card deck thoroughly.
826            shuffleDeck(cards);
827
828            // Deal the player their hand.
829            dealHand();
```

```
830
831             // Show the player their dealt hand.
832             showHand(hand);
833
834             // Ask the player which card/cards to discard.
835             discard(input);
836
837             // Show the player their final chosen hand.
838             showHand(hand);
839
840             // See if the player has a winning hand.
841             evaluateHand(hand);
842
843             // Check to see if the player is out of tokens.
844             if(isBankrupt()) {choice = 0;}
845          }
846        //Keep playing Video Poker unless the player chooses to quit.
847       } while (choice != 0);
848    }
849
850    /* Method: ShowBankroll() simply just displays the player's current
851     *        token balance via the getBankroll() method from the
852     *        Player class.  It returns nothing.
853     */
854
855    public void showBankroll()
856    {
857       System.out.println("\n" +
858          "=========================================================");
859       System.out.println("Your total bankroll is "
860                   + p.getBankroll() + " tokens.");
861       System.out.println("" +
862          "=========================================================");
863    }
864
865    /* Method: ShowGameMenu(Scanner input) just asks the user if he/she
866     *        wants to play one game of Video Poker or "walk from the
867     *        table."  It returns an integer value called "choice" as
868     *        an input parameter to the play() method's "do... while
869     *        (TRUE)" loop.  This governing loop can perpetuate
870     *        an unlimited number of Video Poker games--subject only
871     *        to the user's token balance.
872     */
873
874    public int showGameMenu(Scanner input)
875    {
876       System.out.println("\n" +
877          "=========================================================");
878       System.out.println("Game Menu");
879       System.out.println("" +
880          "=========================================================");
881
```

```
882         showBankroll();
883
884         System.out.println("\nEnter 1 to start/keep playing "
885                         + "Video Poker.");
886         System.out.println("Enter 0 to quit and walk away.");
887         choice = input.nextInt();
888
889         return choice;
890     }
891
892     /* Method: ShowHand(ArrayList<Card> hand) displays the user's
893      *       current 5-card Video Poker playing hand.  It
894      *       returns nothing.
895      */
896
897     public void showHand(ArrayList<Card> hand)
898     {
899         System.out.println("\n" +
900             "=======================================================");
901
902         System.out.print("Your hand: ");
903
904         sort(hand);
905
906         for (int i = 0; i < 5; i++)
907         {
908             if (hand.get(i) != null) {
909                 System.out.print(hand.get(i).toString());
910             }
911
912             if (i + 1 < 5) {
913             System.out.print(" | ");
914             }
915         }
916         System.out.println("\n" +
917             "=======================================================");
918     }
919
920     /* Method: Show_Keep_Or_Discard(int i, Scanner input) just asks
921      *       the user if they want to keep or discard the i'th card
922      *       in their Video Poker playing hand.  It returns an
923      *       integer value back to its parent method, discard().
924      */
925
926     public int show_Keep_Or_Discard(int i, Scanner input)
927     {
928
929         int decision = 0;
930
931         System.out.println("\nKeep or Discard? " +
932                         hand.get(i).toString());
933
```

```
934         System.out.println("Enter 1 to Keep this card.");
935         System.out.println("Enter 0 to Discard this card.");
936 
937         decision = input.nextInt();
938 
939         return decision;
940     }
941 
942     /* Method: ShuffleDeck(Deck cards) tells the user and actually
943      *       performs a very thorough shuffle of the 52-card Video
944      *       Poker playing card deck.  In fact, it may be
945      *       overkill . . .  It returns nothing.
946      */
947     public void shuffleDeck(Deck cards)
948     {
949         System.out.println("\n" +
950             "=======================================================");
951         System.out.println("Shuffling cards . . .");
952         System.out.println("" +
953             "=======================================================");
954 
955         for(int nShuffles = 0; nShuffles < 32767; nShuffles++)
956         {
957             cards.shuffle();
958         }
959     }
960 
961     /* Method: Sort(ArrayList<Card> hand) uses the predefined
962      *       sorthing method from the Java "Collections"
963      *       library class.  It returns nothing.
964      *
965      *       NB: The "public int compareTo(Card c)"
966      *       method, from my user-defined Card class, is
967      *       relied upon by the Collections.sort(hand) Java
968      *       library method to help compare and arrange two
969      *       user-defined Card objects (Cannon).
970      */
971 
972     public void sort(ArrayList<Card> hand)
973     {
974         Collections.sort(hand);
975     }
976 
977     /* Method: SuitType(String letterSuit) is a decomposed method
978      *       created solely to accommodate the overloaded Game()
979      *       constructor's statically-declared "String
980      *       testHand[]" array.  Once parsed into usable input
981      *       parameters, only then can the "checkHand (ArrayList<Card>
982      *       hand)" scoring method be invoked by the testPlay()
983      *       debugging method.  It returns an integer value.
984      */
985 
```

```
986     public int suitType(String letterSuit)
987     {
988        switch(letterSuit) {
989
990              case "C": return 1;
991
992              case "D": return 2;
993
994              case "H": return 3;
995
996              case "S": return 4;
997
998              default: System.out.println("Invalid suit" +
999                    " for said card.");
1000                   return 0;
1001       }
1002    }
1003
1004    /* Method: TestHand_Error_Message(int i) throws an error message if
1005     *       any of the user's inputted string-values (for the i'th
1006     *       card in their testHand[] array) cannot be parsed into a
1007     *       proper Card object-type for the "ArrayList<Card> hand"
1008     *       variable.  It returns nothing.
1009     */
1010
1011    public void testHand_Error_Message(int i)
1012    {
1013       System.out.println("Sorry, card number " + i
1014                     + " is unreadable or invalid.");
1015
1016       System.out.println("Please recheck your input values "
1017                      + "for your testHand[] array "
1018                         + "and try again.");
1019    }
1020
1021    /* Method: TestPlay() is a debugging method created solely for the
1022     *       given PokerTest.java tester file.  It allows the coder
1023     *       to custom-define poker hands they wish to validate with
1024     *       the existing checkHand(ArrayList<Card> hand) Video Poker
1025     *       scoring method.  It returns nothing.
1026     */
1027
1028    public void testPlay()
1029    {
1030
1031       String typeOfHand = "";
1032
1033       int odds = checkHand(hand);
1034
1035       typeOfHand = which_Type_Of_Hand(odds, typeOfHand);
1036
1037       testResult(odds, typeOfHand, hand);
```

```
1038    }
1039
1040    /* Method: TestResult(int odds, String typeOfHand, ArrayList<Card>
1041     *        hand) is a decomposed method of testPlay() just
1042     *        responsible for prompting the programmer of their
1043     *        debugging outcome--per method call, per custom-defined
1044     *        playing hand.  It returns nothing.
1045     */
1046
1047    public void testResult(int odds, String typeOfHand,
1048                    ArrayList<Card> hand)
1049    {
1050       System.out.println("\n" +
1051       "================================================================");
1052
1053       showHand(hand);
1054
1055       System.out.println("\nYour test hand reported a winning "
1056                    + "odds multiplier of " + odds + ".");
1057
1058       System.out.println("Please verify if your hand was "
1059                    + "supposed to be a " + typeOfHand + ".");
1060
1061       System.out.println("" +
1062       "================================================================\n");
1063    }
1064
1065    /* Method: Which_Type_Of_Hand(int odds, String typeOfHand) is a
1066     *        decomposed switch statement the testPlay() parent method
1067     *        used for debugging purposes.  It returns a string value.
1068     */
1069
1070    public String which_Type_Of_Hand (int odds, String typeOfHand)
1071    {
1072       switch(odds) {
1073
1074          case 250: typeOfHand = "Royal Flush";
1075                return typeOfHand;
1076
1077
1078          case 50: typeOfHand = "Straight Flush";
1079                return typeOfHand;
1080
1081          case 25: typeOfHand = "Four of a Kind";
1082                return typeOfHand;
1083
1084          case 6: typeOfHand = "Full House";
1085                return typeOfHand;
1086
1087          case 5: typeOfHand = "Flush";
1088                return typeOfHand;
1089
```

```
1090            case 4: typeOfHand = "Straight";
1091                  return typeOfHand;
1092
1093            case 3: typeOfHand = "Three of a Kind";
1094                  return typeOfHand;
1095
1096            case 2: typeOfHand = "Two Pair";
1097                  return typeOfHand;
1098
1099            case 1: typeOfHand = "One Pair";
1100                  return typeOfHand;
1101
1102            default: typeOfHand = "BUST";
1103                   return typeOfHand;
1104        }
1105    }
1106 }
```

Player.java Download

```java
/* Name: Jinku Jung
 * UNI: dj2598
 * Assignment: Programming Project #4
 * Date: 3/29/2021
 */


/* File: Player.java
 *
 *  Purpose: This is the Player class file for the Video Poker game.
 *  Player.java is responsible for declaring, instantiating, and
 *  executing all money-based functions involving tokens loaded into
 *  the machine, player bets, game winnings, and total tokens accrued.
 *
 *  Instance Variables:
 *
 *     1.) "Bankroll" is an integer that tracks the player's total
 *         tokens accrued in the Video Game, including from the
 *         initial load.
 *
 *     2.) "Bet" is an integer that tracks what the player has
 *         wagered prior to the initial five Video Poker cards
 *         being dealt.  It is used to calculate player winnings
 *         in the "winnings" method as well.
 *
 *  Constructor: Player(), when invoked, just instantiates the two
 *               instance variables of the Player class.
 *
 *  Methods: Player class has five total methods:
 *
 *     1.) public void bets(int amt)
 *
 *     2.) public int getBankroll()
 *
 *     3.) public int getBet()
 *
 *     4.) public void setBankroll(int amt)
 *
 *     5.) public void winnings(int odds)
 *
 */

//Imported from VS Code on 3/29/2021 @ 11:42 AM EST

public class Player {

      //Instance Variables:

   private int bankroll;
```

```
50    private int bet;
51
52       //Constructor:
53
54    public Player()
55    {
56       this.bankroll = 0;
57       this.bet = 0;
58    }
59
60    //Methods (in alphabetical order):
61
62    /* Method: Bets(int amt) is a mutator method that simply
63     *       just sets the value of "bet" to the input variable
64     *       of "amt."  It returns nothing.
65     */
66
67    public void bets(int amt)
68    {
69       this.bet = amt;
70    }
71
72    /* Method: GetBankroll() is an accessor method that just
73     *       returns the current value of the instance variable
74     *       "bankroll" as an integer value.
75     */
76
77    public int getBankroll()
78    {
79       return bankroll;
80    }
81
82    /* Method: GetBet() is an accessor method that just
83     *       returns the current value of the instance variable
84     *       "bet" as an integer value.
85     */
86
87    public int getBet()
88    {
89       return bet;
90    }
91
92    /* Method: SetBankroll(int amt) is a mutator method which
93     *       increments "bankroll" by the integer input parameter
94     *       "amt."  It returns nothing.
95     */
96
97    public void setBankroll(int amt) {
98       this.bankroll += amt;
99    }
100
101   /* Method: Winnings(int odds) is a mutator method which
```

```
102     *       increments "bankroll" by the amount the player just
103     *       won, where his/her winnings = (bet * odds).  It returns
104     *       nothing.
105     */
106
107    public void winnings(int odds)
108    {
109       this.bankroll += bet * odds;
110    }
111 }
112
113
114
115
```

PokerTest.java Download

```
1  public class PokerTest{
2
3     // this class must remain unchanged
4     // your code must work with this test class
5
6     public static void main(String[] args){
7        if (args.length<1){
8           Game g = new Game();
9           g.play();
10       }
11       else{
12          Game g = new Game(args);
13          g.testPlay();
14       }
15    }
16
17 }
18
```

readMe.txt Download

```
README.txt
Name: Jinku Jung
UNI: dj2598
Assignment: Programming Project #4
Date: 3/29/2021

================================================================================
Part 1: Single-player Video Poker game.
================================================================================

Once the single-player Game() class is declared and instantiated, its play()
method (containing the core runtime code of the Video Poker game) is launched.
The user is then promped to enter the number of tokens they wish to add to the
game's token balance integer variable, "bankroll."  After the player then
places their bet of 1-5 tokens, a single game of Video Poker ensues.

   1.) The deck is then well-shuffled (32,767 times!).

      NB: Credit for this shuffle algorithm used in my Video Poker game goes
         to Professor Shun Yan Cheung's "DeckOfCards" Java class definition
         from his CS 170 class at Emory University:

         <http://www.mathcs.emory.edu/~cheung/Courses/170/Syllabus/10/
         deck-of-cards.html>.

   2.) Then the initial 5 cards are dealt via the dealHand() method.  Cards
      chosen are taken from the Deck object "cards," an immutable array of
      52 Card objects, and placed into an ArrayList variable called "hand"
      inside the actual Video Poker code.

   3.) The player's hand is then outputted to the screen via the
      showHand(hand) method.

   4.) Now, the player is given to option to keep or discard each card from
      his original Video Poker hand.  He/she must choose which cards to
      discard & replace before his new Video Poker hand is displayed via
      another call to the showHand(hand) method inside play().

   5.) Finally, the Video Poker hand is scored via the evaluateHand(hand)
      method inside play().  Many sub-methods are called to determine if
      the player's hand is indeed a winner, what his payout should be
      based upon his initial bet and the winning odds multiplier (returned
      from checkHand(hand)), and to output the game results to display to
      the user.

   6.) If the player has run out of tokens, he is declared bankrupt and
      kicked out of the game.

```

```
   7.) However, if the player has at least one token left in his bankroll
       balance, he/she can choose to play Video Poker again or walk from
       the table and exit the game.

NB: A big thanks to my old CS106A Professor Eric Roberts and Section TA
   Keith Richards for being so hard on grading my programming assignments
   way back when . . .  I hope my current code and documentation in this
   Video Poker assignment reflect upon their teachings well:

   <http://web.stanford.edu/class/archive/cs/cs106a/cs106a.1142/handouts/
   17-coding-style.pdf>.


================================================================================
Part 2: Video Poker debugging mode, given a statically-declared poker hand.
================================================================================

With the overloaded version of the Game() constructor, the Video Poker game's
checkHand(ArrayList<Card> hand) method can be exclusively invoked via
testPlay(), given a statically declared String[] array of cards in two/three
character form as an input parameter.  This is useful to validate each and
every winning hand sub-method and ensure accurate odds pay out at the end of
each Video Poker game.  As directed, the PokerTest.java tester file was left
unaltered and helped and actually helped immensely while fine-tuning the
final program's code.

--------------------------------------------------------------------------------

Final Thoughts: Wow--the time I spent creating the comment code/documentation
                for Video Poker nearly equaled to time I spend coding the
                project!  In IT/CS work, I have this really naive tendency to
                underestimate the actual amount of time a project will take
                to complete.  Coding was the fun part; commenting was the
                painful part.  But, as with all STEM work, what good is
                an answer with a lousy explanation?

                                                --Jinku Jung

--------------------------------------------------------------------------------
```